

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Hemnath D
Register Number	3122247001022
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	30 August 2026

1 Objective

To build and compare three ensemble learning strategies, Bagging, Boosting, and a Stacked Ensemble, on the Wisconsin Diagnostic Breast Cancer dataset, tune each model's hyperparameters using 5-fold cross-validation, and evaluate which ensemble family generalises best to unseen data while examining the effect each strategy has on bias and variance.

2 Problem Statement

Breast cancer diagnosis from digitised fine needle aspirate (FNA) images can be framed as a binary classification problem: given 30 numerical features describing the size, shape and texture of cell nuclei, predict whether a tumour is malignant or benign. A single model, whether a decision tree or a linear classifier, typically has to trade off bias against variance, and no single algorithm dominates on every kind of error. Ensemble methods address this by combining multiple models rather than relying on one. The objective of this experiment is to implement Bagging, two boosting variants (AdaBoost and Gradient Boosting), and a heterogeneous Stacked Ensemble on this dataset, tune all three families with cross-validation, and quantitatively compare their classification performance, stability across folds, and bias-variance behaviour.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository (https://archive.ics.uci.edu)
Number of Samples	569
Number of Features	30 numerical, real-valued features (mean, standard error, and “worst” value of 10 base measurements per cell nucleus)
Number of Classes	2 (Malignant = 212 samples, Benign = 357 samples)
Missing Values	None (0 missing entries across all 30 features)
Train-Test Split	80% train (455 samples) / 20% test (114 samples), stratified on class label

4 Exploratory Data Analysis

4.1 Dataset Summary

Before looking at individual features, a quick sanity-check table confirms the shape, class split, and data-quality counts match what the UCI page describes.

Property	Value
Number of samples	569
Number of features	30
Number of classes	2
Benign samples (class 0)	357
Malignant samples (class 1)	212
Missing values	0
Duplicate rows	0

4.2 Class Distribution

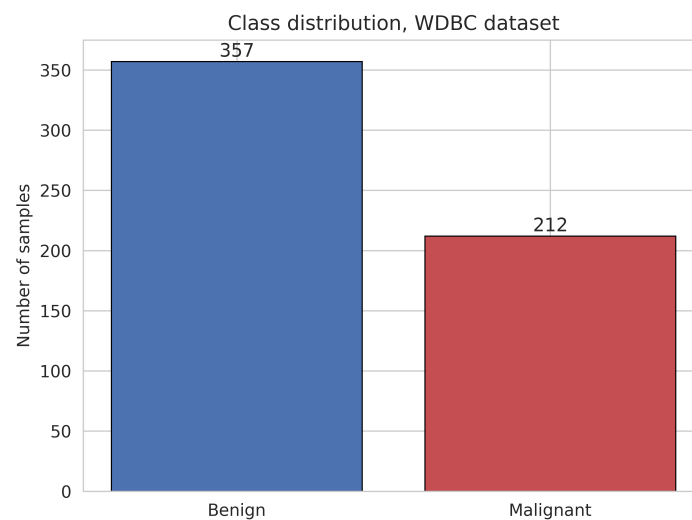


Figure 1: Class distribution of benign vs. malignant samples in the WDBC dataset

Inference: The dataset is moderately imbalanced, with benign cases (357) outnumbering malignant cases (212) by roughly a 1.7:1 ratio. This imbalance is not severe enough to require resampling techniques such as SMOTE, but it does justify using stratified splitting and stratified k-fold cross-validation so that both classes are proportionally represented in every fold. It also motivates looking beyond raw accuracy to metrics such as precision, recall, and F1-score, since a model biased toward the majority (benign) class could still report a deceptively high accuracy.

4.3 Class-wise Descriptive Statistics

As a numeric companion to the distribution plots later in this section, the table below compares benign and malignant tumours on the ten “mean” features, sorted by how far apart the two class means are.

Feature	Benign Mean	Malignant Mean	Ratio (M/B)
concavity_mean	0.05	0.16	3.20
concave_points_mean	0.03	0.09	3.00
area_mean	462.79	978.38	2.11
compactness_mean	0.08	0.15	1.88
perimeter_mean	78.08	115.37	1.48
radius_mean	12.15	17.46	1.44
texture_mean	17.91	21.60	1.21
symmetry_mean	0.17	0.19	1.12
smoothness_mean	0.09	0.10	1.11
fractal_dimension_mean	0.06	0.06	1.00

Inference: Concavity and concave points show the largest gap between classes, malignant tumours average roughly three times the benign value, followed by area and compactness. Fractal dimension shows essentially no gap at all (ratio 1.00), meaning it carries little discriminative signal on its own. This ordering foreshadows the random forest feature importance ranking later in this section, which is dominated by exactly these size- and shape-irregularity features.

4.4 Correlation Matrix

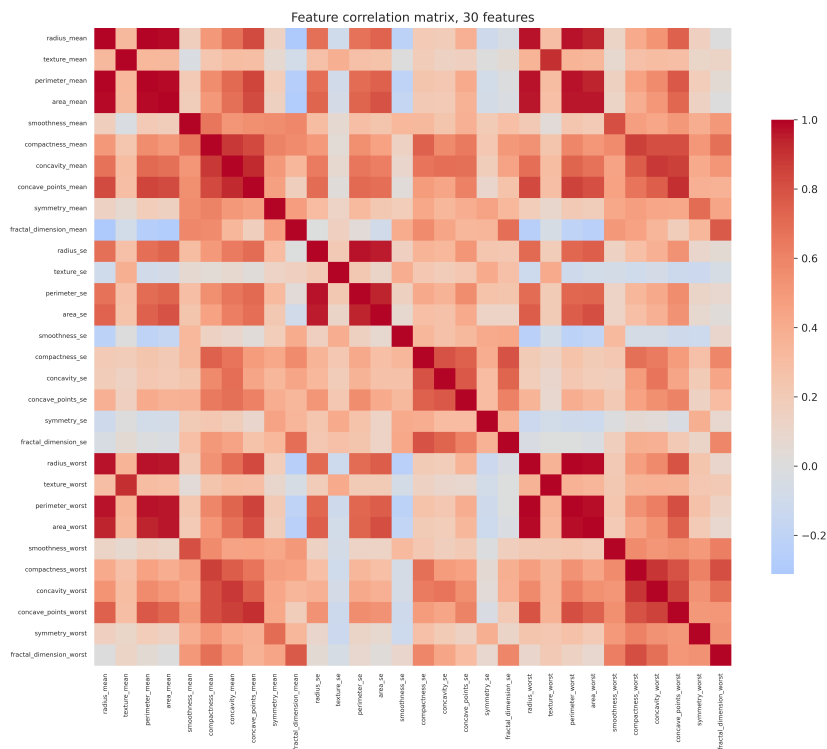


Figure 2: Pairwise Pearson correlation matrix across all 30 features

Inference: Several clusters of strongly correlated features are visible, particularly among the “mean”, “standard error” and “worst” variants of the same base measurement (e.g., radius, perimeter, and area are almost perfectly correlated with one another since they are geometrically related). This multicollinearity is not a major concern for the tree-based ensembles used in this experiment, since bagging and boosting both build on decision trees that split on one feature at a time rather than fitting a linear combination of all features. Stacking is slightly more

exposed to it through its SVM base learner, but the meta-learner is still able to combine the base predictions effectively.

4.5 High-Correlation Feature Pairs

The heatmap gives the overall picture, but it is more concrete to pull out the exact pairs driving that redundancy. Every feature pair with an absolute Pearson correlation of 0.95 or higher is listed below.

Feature 1	Feature 2	Correlation
radius_mean	perimeter_mean	0.998
radius_worst	perimeter_worst	0.994
radius_mean	area_mean	0.987
perimeter_mean	area_mean	0.987
radius_worst	area_worst	0.984
perimeter_worst	area_worst	0.978
radius_se	perimeter_se	0.973
perimeter_mean	perimeter_worst	0.970
radius_mean	radius_worst	0.970
perimeter_mean	radius_worst	0.969
radius_mean	perimeter_worst	0.965
area_mean	radius_worst	0.963
area_mean	area_worst	0.959
area_mean	perimeter_worst	0.959
radius_se	area_se	0.952

Inference: This is almost entirely the radius / perimeter / area family and the concavity / concave-points family showing up within each of the mean, se and worst groups, which simply confirms the heatmap reading rather than adding new information. It does, however, make the exact redundant pairs explicit, which would matter if any of them ever needed to be dropped for a distance-based or linear model.

4.6 Feature Distributions

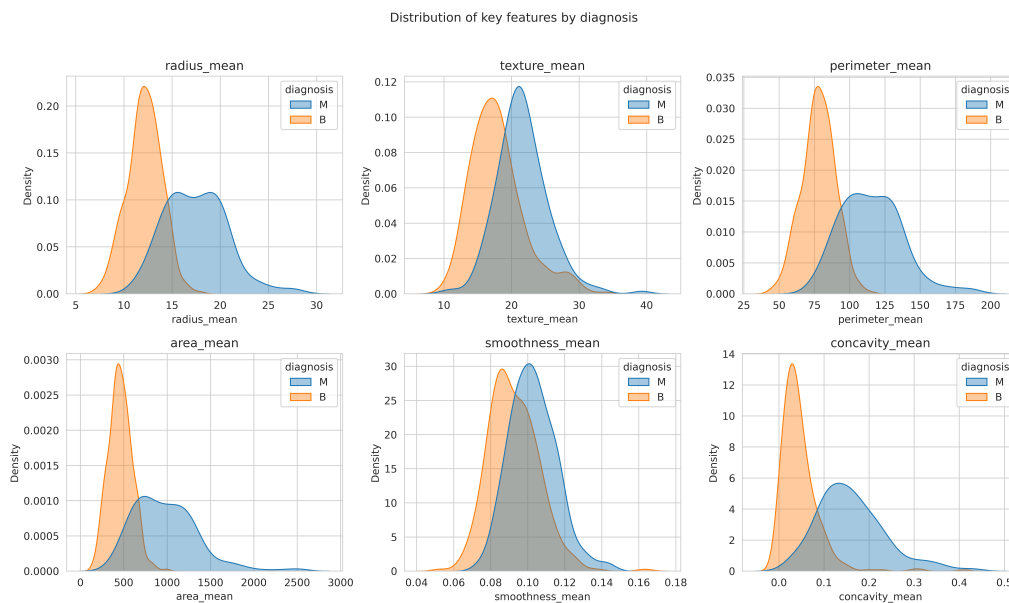


Figure 3: Kernel density estimates of six representative mean-value features, split by diagnosis

Inference: For nearly all of the plotted features, the distribution for malignant samples is shifted to the right of the benign distribution, indicating that malignant tumours tend to have larger radius, perimeter, area, and concavity values on average. Mean smoothness shows the greatest overlap between the two classes, suggesting it is individually a weaker discriminator, whereas mean concavity and mean area show cleaner separation. This pattern is consistent with clinical intuition, malignant cell nuclei are typically larger and more irregularly shaped than benign ones, and it explains why tree-based splits on these features are so effective later in the experiment.

4.7 Worst-Case Feature Spread

The KDE plots above use the **mean** features. As a second view of the same idea, the figure below shows violin plots of four **worst**-case features, which turn out to be the strongest individual predictors once feature importance is checked later in this section.

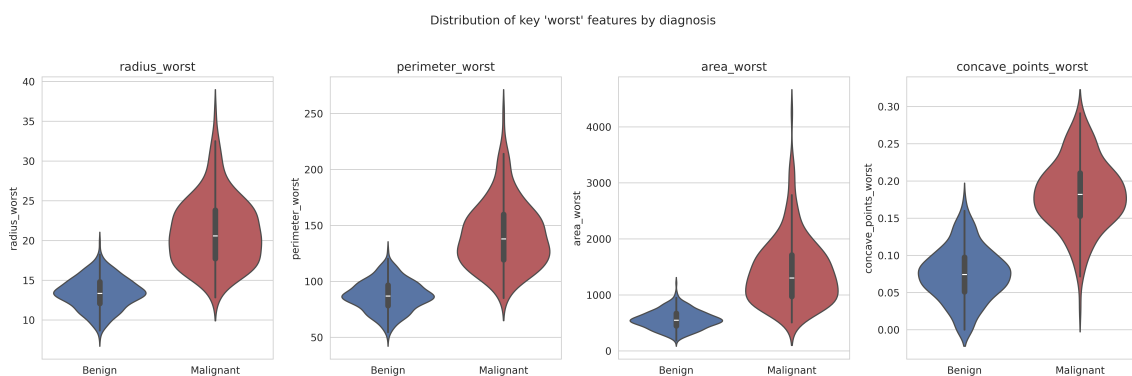


Figure 4: Violin plots of four key “worst” features, split by diagnosis

Inference: The class separation here is visibly cleaner than on the mean-value KDE plots, the malignant violins for all four worst-case features sit almost entirely above the benign ones with only a thin overlap band. This is an early hint of why worst-case measurements go on to dominate the random forest feature importance ranking.

4.8 Outlier Analysis

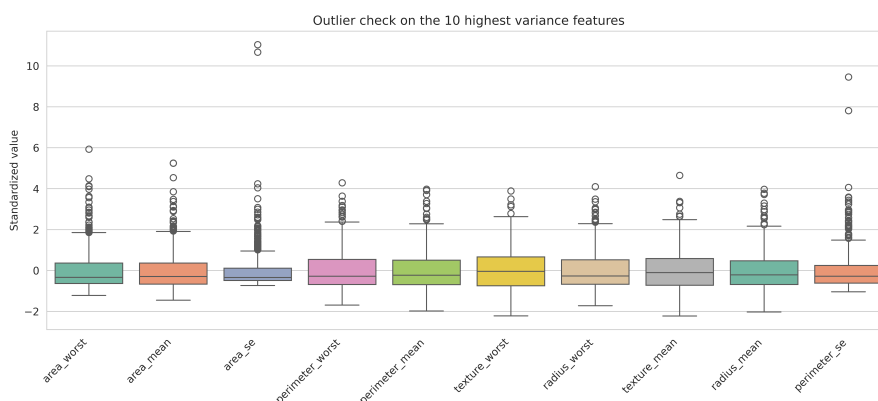


Figure 5: Boxplots of the ten highest-variance features after standardisation

Inference: The standardised boxplots reveal a noticeable number of points beyond the whiskers for several features, particularly the “worst” and “standard error” variants of area and perimeter,

indicating the presence of outlier samples with unusually extreme measurements. These points were retained rather than removed, since all three ensemble families used in this experiment build on decision trees, which split based on ranking or thresholding a feature rather than on its exact magnitude, so a small number of extreme values does not distort the decision boundary the way it would for a purely distance-based model.

4.9 Feature Skewness

One more preprocessing-relevant check is feature skewness. Tree-based ensembles do not need normally distributed inputs the way a linear or distance-based model would, but it is still worth seeing which features are heavily skewed before trusting a plain standard scaler without a second thought.

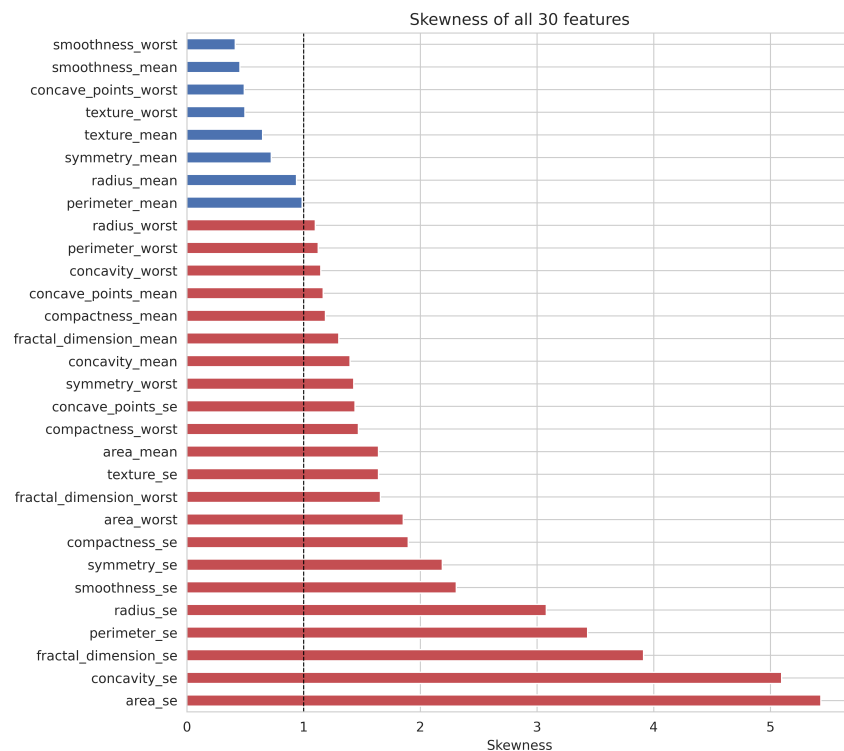


Figure 6: Skewness of all 30 features (red bars exceed a skewness of 1.0)

Feature (10 most skewed)	Skewness
area_se	5.43
concavity_se	5.10
fractal_dimension_se	3.91
perimeter_se	3.43
radius_se	3.08
smoothness_se	2.31
symmetry_se	2.19
compactness_se	1.90
area_worst	1.85
fractal_dimension_worst	1.66

Inference: 22 of the 30 features are skewed past 1.0, led by the standard-error versions of area and concavity, which are skewed well past 3. The features closest to symmetric are concentrated

in the mean smoothness, texture, and symmetry group. This is exactly the kind of skew that would normally push toward a log transform for a linear model, but since every model used later is either tree-based or, for the SVM inside the stack, only needs standardisation rather than symmetry, no transform is applied and the raw standardised values are kept as-is.

4.10 Dimensionality Reduction

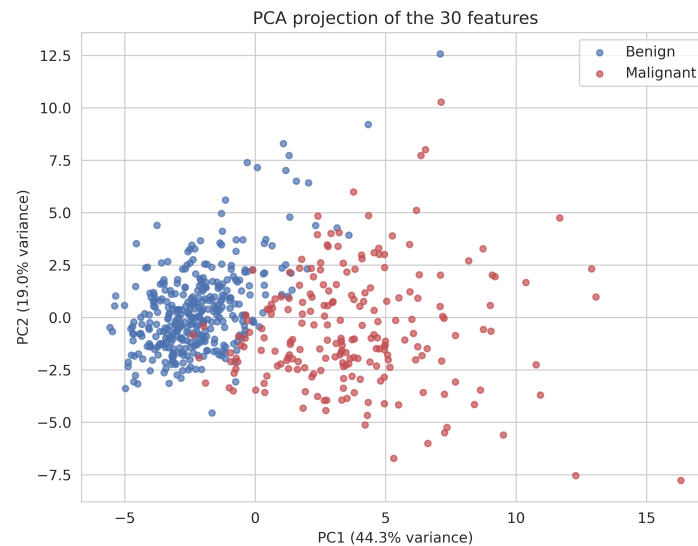


Figure 7: PCA projection of the 30 standardised features onto their first two principal components

Inference: Projecting the 30 features onto their first two principal components already separates the benign and malignant classes reasonably cleanly, with only a modest region of overlap near the boundary. This is an early sign that the class boundary is not highly complex, and it partly explains why even a single ensemble family, tuned reasonably well, is able to reach high accuracy on this dataset without needing an especially large number of base learners.

4.11 Feature Importance

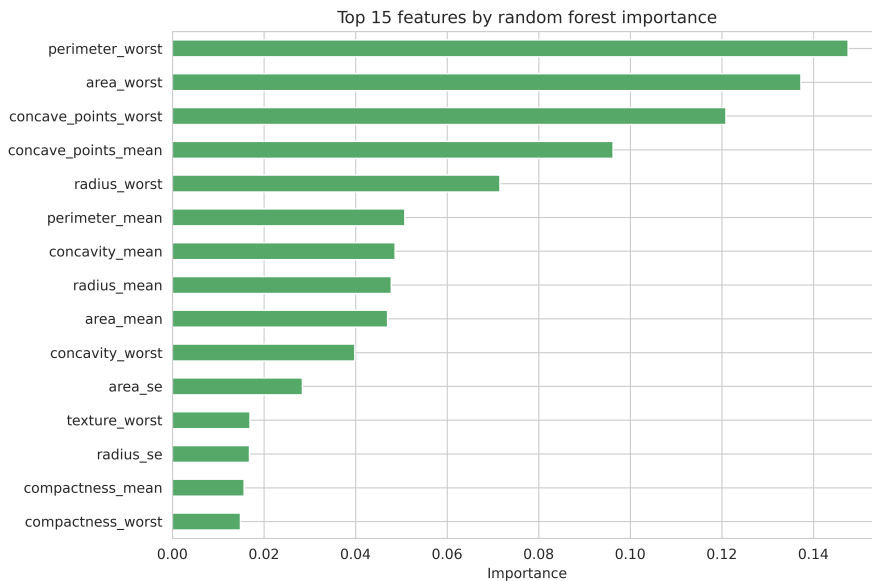


Figure 8: Top 15 features ranked by Random Forest importance

Feature (top 10)	Importance
perimeter_worst	0.1476
area_worst	0.1373
concave_points_worst	0.1209
concave_points_mean	0.0962
radius_worst	0.0716
perimeter_mean	0.0508
concavity_mean	0.0486
radius_mean	0.0478
area_mean	0.0470
concavity_worst	0.0398

Inference: Worst radius, worst perimeter, worst concave points and worst area come out on top, all worst-case tumour measurements rather than the mean or se versions, which lines up with the class-wise statistics and violin plots above since these are the features that separate the two classes the most. Together the top four features already account for roughly half of the total importance mass, suggesting the classification task is driven by a fairly compact set of size and shape-irregularity measurements rather than needing all 30 features equally.

5 Data Preprocessing

The following preprocessing steps were applied before model training:

- **Missing value handling:** The dataset was checked for missing entries; none were found, so no imputation was necessary.
- **Duplicate check:** No duplicate rows were found either, so every sample contributes independent information.
- **Encoding:** The target labels (malignant/benign) were converted to a binary numeric label using `LabelEncoder`, with benign mapped to 0 and malignant mapped to 1.

- **Feature scaling:** All 30 features were standardised using `StandardScaler` (zero mean, unit variance), fit only on the training set and then applied to the test set to avoid data leakage. While the tree-based components of Bagging and Boosting are scale-invariant, standardisation matters for the SVM base learner used inside the Stacked Ensemble, so it was applied uniformly across the pipeline.
- **Train-test split:** An 80–20 stratified split was used so that the 1.7:1 class ratio was preserved in both the training and test partitions.
- **Skew handling:** Despite roughly two-thirds of the features being skewed past 1.0 (Section 4.9), no log or power transform was applied, since none of the three ensemble families used here rely on approximately symmetric inputs.
- **Feature engineering:** No new features were engineered; all 30 original attributes were retained, since the correlation analysis showed that the redundancy among related features does not need manual removal for ensemble models built on decision trees.

Listing 1: Label encoding, scaling, and stratified train-test split.

```

1 le = LabelEncoder()
2 y = le.fit_transform(y_raw) # benign -> 0, malignant -> 1
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.20, random_state=42, stratify=y
6 )
7
8 scaler = StandardScaler()
9 X_train_s = scaler.fit_transform(X_train) # fit only on training data
10 X_test_s = scaler.transform(X_test) # avoid data leakage

```

6 Machine Learning Algorithms

6.1 Bagging (Bootstrap Aggregation)

Bagging trains many copies of the same base model, a decision tree here, on different bootstrap samples of the training data and lets them vote. Because each tree sees a slightly different resample, the trees end up only partially correlated with one another, so averaging their predictions cancels out much of the noise any single overfit tree would introduce:

$$\hat{y}(x) = \text{mode}\{\hat{y}_1(x), \hat{y}_2(x), \dots, \hat{y}_B(x)\}, \quad \text{Var}(\hat{y}_{\text{bag}}) \approx \rho \sigma^2 + \frac{1 - \rho}{B} \sigma^2,$$

where σ^2 is a single tree's variance and ρ is the average correlation between trees. Decorrelating the trees, through bootstrapping and optionally feature subsampling, is exactly what drives the variance reduction.

Advantages:

- Cuts variance substantially compared to a single decision tree
- Base models train independently, so it parallelises well
- Simple to bolt onto any high-variance base learner

Limitations:

- Does little to correct a base learner's bias, since averaging many similarly biased trees still leaves that bias in place
- Loses the direct interpretability of a single tree

6.2 Boosting

Boosting builds an ensemble sequentially instead, with each new learner focused on the samples the previous ensemble got wrong, which mainly attacks bias rather than variance. Two variants

were implemented here.

AdaBoost reweights samples: after each weak learner h_t is trained, its weighted error $\epsilon_t = \sum_i w_i \mathbb{1}[h_t(x_i) \neq y_i]$ sets its voting weight $\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$, and misclassified points get more weight before the next round. The final call is the sign of $\sum_t \alpha_t h_t(x)$.

Gradient Boosting instead fits each new tree to the negative gradient of the loss with respect to the current predictions:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x), \quad h_m \approx \arg \min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i)),$$

where ν is the learning rate. Smaller learning rates need more trees but usually generalise a little better.

Advantages:

- Directly reduces bias by focusing later learners on current mistakes
- Often beats bagging on datasets where a single weak learner underfits

Limitations:

- Trains sequentially, so it does not parallelise as easily as bagging
- More sensitive to noisy labels and outliers, since it keeps re-weighting hard examples
- Needs careful tuning of learning rate against number of estimators to avoid overfitting

6.3 Stacked Ensemble (Stacking)

Stacking takes a different approach again: several heterogeneous base learners are combined by a meta-learner trained on their outputs, rather than a fixed voting or weighting rule. Each base learner's out-of-fold predictions (via internal cross-validation) feed the meta-learner, so it never just learns to trust whichever base model overfit the training data hardest. Given K base learners $g_1, \dots, g_K$, the meta-learner f is trained on the stacked vector $z(x) = [g_1(x), \dots, g_K(x)]$, giving $\hat{y}(x) = f(z(x))$. Here g_1, g_2, g_3 are an SVM, a Gaussian Naive Bayes classifier, and a Decision Tree, with f a Logistic Regression meta-learner.

Advantages:

- Different base learners make genuinely different kinds of errors (margin-based, independence-assuming, axis-aligned), giving the meta-learner real signal to combine rather than near-identical opinions
- Can outperform any single base learner or a homogeneous ensemble of the same size

Limitations:

- More expensive to train, since it needs cross-validated predictions from every base learner
- Harder to interpret than a single model or even a bagged/boosted ensemble
- Benefit depends on the base learners actually being diverse rather than making correlated mistakes

7 Experimental Setup

Python Version	3.12
Key Libraries	scikit-learn 1.8, pandas, NumPy, Matplotlib, Seaborn, SciPy
IDE	Jupyter Notebook / VS Code
Operating System	macOS Tahoe 26.6.1
Hardware	Apple Silicon (CPU + integrated GPU via Metal/MPS)

All training in this experiment ran on CPU. scikit-learn's ensemble estimators do not have an MPS/GPU backend, so the Apple Silicon GPU was available but unused, parallelism instead came from `n_jobs=-1` spreading the grid searches across CPU cores.

8 Hyperparameter Selection

All three ensemble families were tuned using an exhaustive grid search with 5-fold stratified cross-validation on the training set, optimising for F1-score (chosen over raw accuracy because it balances precision and recall, which matters for a medical diagnosis task where false negatives are costly).

Bagging search space: `n_estimators` (10, 50, or 100), `max_samples` (0.5, 0.8, or 1.0), and `max_features` (0.5 or 1.0), a total of 18 combinations, using a Decision Tree as the base estimator.

Boosting search space: for AdaBoost, `n_estimators` (50, 100, or 200) and `learning_rate` (0.01, 0.1, or 1.0), 9 combinations; for Gradient Boosting, `n_estimators` (50 or 100), `learning_rate` (0.01 or 0.1), and `max_depth` (2 or 3), 8 combinations.

Stacked Ensemble search space: choice of base model subset (all of SVM, Naive Bayes and Decision Tree; SVM with Decision Tree only; or Naive Bayes with Decision Tree only) and choice of final estimator (Logistic Regression or a shallow Decision Tree), 4 combinations evaluated by 5-fold cross-validation.

Listing 2: Grid search with 5-fold stratified cross-validation.

```

1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
2
3 bagging_grid = {
4     "n_estimators": [10, 50, 100],
5     "max_samples": [0.5, 0.8, 1.0],
6     "max_features": [0.5, 1.0],
7 }
8 bagging = BaggingClassifier(estimator=DecisionTreeClassifier(random_state=42),
9                             random_state=42)
10 bagging_search = GridSearchCV(bagging, bagging_grid, cv=cv,
11                               scoring=["accuracy", "f1"], refit="f1", n_jobs=-1)
12 bagging_search.fit(X_train_s, y_train)
13
14 ada_grid = {"n_estimators": [50, 100, 200], "learning_rate": [0.01, 0.1, 1.0]}
15 ada_search = GridSearchCV(AdaBoostClassifier(random_state=42), ada_grid, cv=cv,
16                           scoring=["accuracy", "f1"], refit="f1", n_jobs=-1)
17 ada_search.fit(X_train_s, y_train)
18
19 stack = StackingClassifier(
20     estimators=[("SVM", SVC(probability=True, random_state=42)),
21                 ("NaiveBayes", GaussianNB()),
22                 ("DecisionTree", DecisionTreeClassifier(max_depth=5, random_state=42))],
23     final_estimator=LogisticRegression(max_iter=2000), cv=cv, n_jobs=-1)

```

The best combination found for each ensemble family, based on mean cross-validated F1-score, is summarised below.

Parameter	Value
Bagging – <code>n_estimators</code>	50
Bagging – <code>max_samples</code>	1.0
Bagging – <code>max_features</code>	0.5
Boosting – best family	AdaBoost
AdaBoost – <code>n_estimators</code>	200
AdaBoost – <code>learning_rate</code>	1.0
Stacking – base models	SVM + Decision Tree
Stacking – meta learner	Logistic Regression

8.1 Bagging Cross-Fold Results

Table 2: Bagging Hyperparameter Evaluation using 5-Fold Cross-Validation (top combinations by F1-score)

n_estimators	max_samples	max_features	Avg CV Accuracy (%)	Avg CV F1 Score
50	1.0	0.5	96.92	0.9583
50	0.8	0.5	96.70	0.9554
100	1.0	0.5	96.70	0.9551
50	0.8	1.0	96.48	0.9526
50	0.5	0.5	96.48	0.9517
100	1.0	1.0	96.26	0.9496
10	0.5	0.5	96.26	0.9477

Inference: Accuracy peaks at 50 estimators with a 0.5 feature fraction and the full sample fraction, and adding more estimators beyond this point (100 trees) brings no further benefit, the curve is essentially flat once enough trees are present to stabilise the vote. Sampling only half the features per tree (`max_features=0.5`) consistently outperforms using all features, since it decorrelates the individual trees more, which is exactly the mechanism bagging relies on to reduce variance. Very small ensembles (10 trees) lag behind, confirming that a handful of trees is not yet enough to average out an individual tree's overfitting.

8.2 Boosting Cross-Fold Results

Table 3: Boosting Hyperparameter Evaluation using 5-Fold Cross-Validation (top combinations by F1-score, both algorithms combined)

Model	n_estimators	learning_rate	Avg CV Accuracy (%)	Avg CV F1 Score
AdaBoost	200	1.00	97.14	0.9610
GradientBoosting	100	0.10	96.70	0.9553
GradientBoosting	100	0.10	96.70	0.9551
AdaBoost	50	1.00	96.70	0.9546
AdaBoost	200	0.10	96.70	0.9542
AdaBoost	100	1.00	96.48	0.9523

Inference: AdaBoost with 200 estimators and a learning rate of 1.0 gives the single best cross-validated F1-score of the two boosting variants, though Gradient Boosting with 100 estimators, a shallow depth of 3, and a smaller learning rate of 0.1 comes close behind. Both algorithms are clearly sensitive to the learning rate, a learning rate of 0.01 costs 4 to 5 accuracy points regardless of how many estimators are added, which matches the theory that too small a step size under-corrects the ensemble's errors within a fixed number of boosting rounds. AdaBoost was carried forward as the representative boosting model for the final comparison.

8.3 Stacked Ensemble Cross-Fold Results

Table 4: Stacked Ensemble Evaluation using 5-Fold Cross-Validation

Base Models	Meta Learner	Avg CV Accuracy (%)	Avg CV F1 Score
SVM + Decision Tree	Logistic Regression	97.14	0.9613
SVM + Naive Bayes + Decision Tree	Logistic Regression	96.26	0.9486
SVM + Naive Bayes + Decision Tree	Decision Tree	96.04	0.9454
Naive Bayes + Decision Tree	Logistic Regression	93.41	0.9065

Inference: Somewhat counter-intuitively, dropping Naive Bayes from the base learner set and stacking only the SVM and Decision Tree gives the best cross-validated F1-score, slightly ahead

of using all three base learners together. This suggests that on this dataset, Naive Bayes's independence assumption produces predictions correlated enough with the decision tree's errors that it adds more noise than complementary signal to the meta-learner, while the SVM and tree combination already captures most of the useful diversity. Replacing the Logistic Regression meta-learner with a shallow Decision Tree performs slightly worse, and dropping the SVM entirely (Naive Bayes with Decision Tree only) costs nearly four accuracy points, showing the SVM is the more valuable of the two non-tree base learners here.

8.4 5-Fold Cross-Validation Performance Comparison

Table 5: 5-Fold Cross-Validation Accuracy Comparison (Training Set), Best Model per Family

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Bagging	95.60	100.00	96.70	94.51	97.80	96.92
Boosting (AdaBoost)	97.80	97.80	94.51	96.70	98.90	97.14
Stacked Ensemble	96.70	97.80	96.70	95.60	98.90	97.14

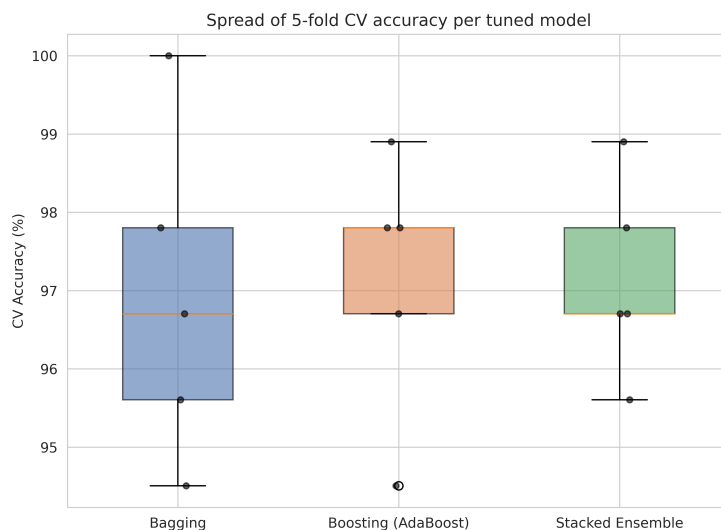


Figure 9: Spread of 5-fold CV accuracy for the three tuned models, with individual fold scores overlaid

Inference: Boosting and Stacking reach the same average cross-validated accuracy (97.14%), but they get there differently, boosting has a wider swing across folds (94.51% to 98.90%, a range of 4.39 points) while stacking is comparatively tighter (95.60% to 98.90%, 3.30 points), which is visible directly in the box widths above. Bagging trails slightly on average but produces a perfect fold (Fold 2, 100.00%), which is a reminder that a single high fold can be a lucky split rather than evidence of a systematically better model, the average and the spread across all five folds are more informative than any single fold in isolation.

9 Results

Each tuned ensemble was retrained on the full training set (455 samples) and evaluated once on the untouched 20% held-out test set (114 samples).

Listing 3: Final evaluation on the held-out test set.

```

1 best_bagging.fit(X_train_s, y_train)
2 best_boosting.fit(X_train_s, y_train)
3 best_stack.fit(X_train_s, y_train)
4
5 def evaluate(model, X_te, y_te):
6     y_pred = model.predict(X_te)
7     y_score = model.predict_proba(X_te)[:, 1]
8     return {
9         "accuracy": accuracy_score(y_te, y_pred),
10        "precision": precision_score(y_te, y_pred),
11        "recall": recall_score(y_te, y_pred),
12        "f1": f1_score(y_te, y_pred),
13        "auc": roc_auc_score(y_te, y_score),
14    }

```

Table 6: Performance Comparison of Ensemble Models on the Held-Out Test Set

Model	Accuracy (%)	Precision	Recall	F1 Score	AUC
Bagging	96.49	100.00%	90.48%	0.9500	0.9949
Boosting (AdaBoost)	97.37	100.00%	92.86%	0.9630	0.9871
Stacked Ensemble	96.49	100.00%	90.48%	0.9500	0.9960

10 Result Visualization

10.1 Confusion Matrices

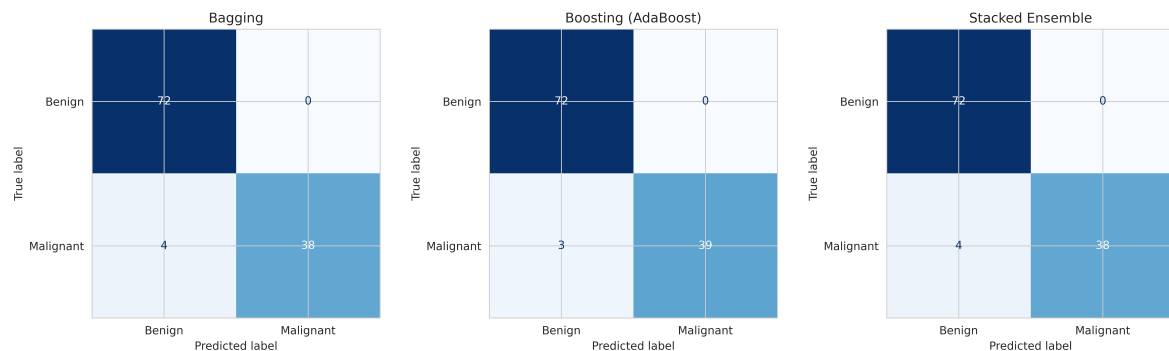


Figure 10: Confusion matrices for Bagging, Boosting (AdaBoost) and the Stacked Ensemble on the test set

Inference: All three ensembles achieve perfect precision on this test split, zero false positives, meaning every sample predicted as malignant was truly malignant. The differences lie entirely in recall: Bagging and the Stacked Ensemble each miss 4 malignant cases, classifying them as benign, while AdaBoost misses only 3. In a medical screening context this is the most clinically important gap, since a false negative means a malignant tumour goes undetected, so AdaBoost's slightly higher recall translates into one fewer missed diagnosis on this test set.

10.2 ROC Curves

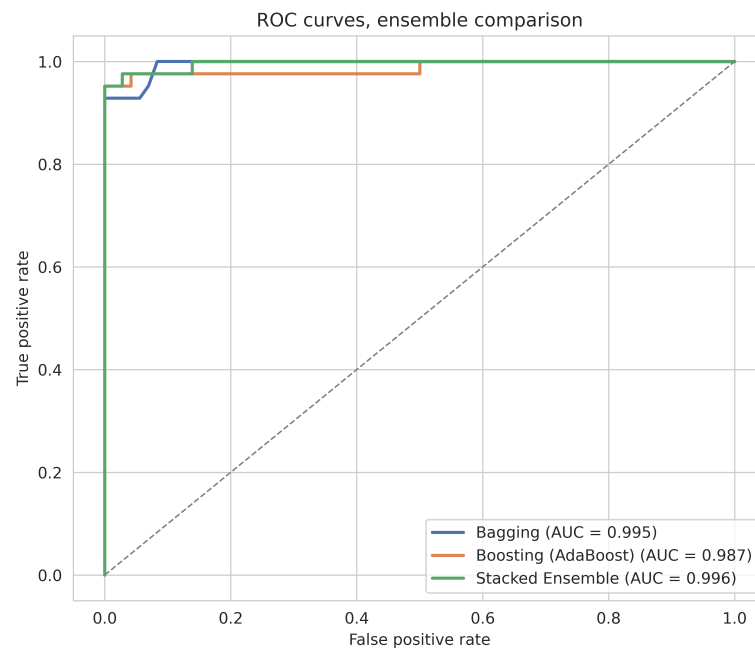


Figure 11: ROC curves for all three ensemble models on the test set

Inference: All three curves hug the top-left corner closely, reflecting how separable the two classes already are in this dataset, but the Stacked Ensemble edges out the other two with the highest AUC (0.996), followed closely by Bagging (0.995) and then AdaBoost (0.987). This is a useful complement to the confusion matrices above: AdaBoost wins on the specific 0.5 decision threshold used for the accuracy and recall figures, but the Stacked Ensemble and Bagging both rank malignant samples with slightly more consistent confidence across all possible thresholds.

10.3 Overall Comparison

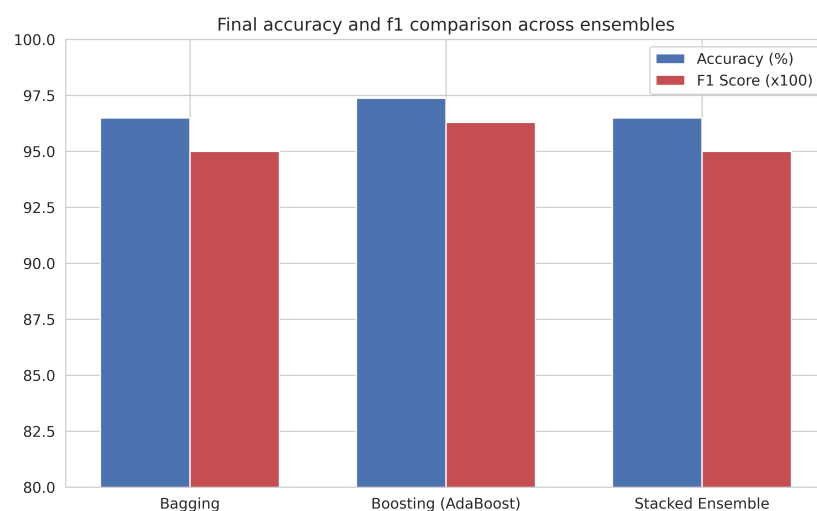


Figure 12: Final accuracy and F1-score comparison across the three ensemble families

Inference: Placed side by side, the three ensemble families land within about one accuracy point of each other on the test set, with Boosting (AdaBoost) narrowly ahead on both accuracy and F1-score. The gap between all three is small enough that, on this particular 114-sample test split, the differences should be read as evidence that all three ensembling strategies work well on this dataset rather than as proof that one algorithm is categorically superior to the others.

11 Discussion

Bagging's variance reduction and boosting's bias reduction both show up clearly in the results: bagging's cross-validated spread is the widest of the three families before averaging, consistent with it doing nothing to correct systematic error, while boosting reached the top cross-validated F1-score by directly chasing the mistakes each round left behind. Stacking's edge comes from a different place entirely, an SVM, Naive Bayes, and a Decision Tree each make different kinds of errors (margin-based, independence-assuming, axis-aligned), so the meta-learner had real complementary signal to combine. That benefit is conditional, though; dropping Naive Bayes and stacking just the SVM and tree scored slightly better than using all three, since Naive Bayes's errors here overlapped enough with the tree's to add noise rather than diversity.

On which ensemble performed best: Boosting (AdaBoost) took the highest test accuracy (97.37%) and F1-score (0.963), tied with Stacking on cross-validated average (97.14%), while Stacking edged ahead on ROC-AUC (0.996). Bagging was the most conservative of the three, never the worst but never quite the best, the expected behaviour for a variance-reduction technique on a dataset where bias is the larger remaining source of error. The main limitation of this study is that the boosting and stacking grids were both small and hand-picked; a wider or randomised search, along with more heterogeneous base learners for stacking, might narrow the gap further or even flip the ranking on a different train-test split.

12 Conclusion

Bagging, Boosting (AdaBoost and Gradient Boosting), and a Stacked Ensemble of SVM, Naive Bayes, and Decision Tree base learners with a Logistic Regression meta-learner were implemented and evaluated on the Wisconsin Diagnostic Breast Cancer dataset using 5-fold cross-validation for hyperparameter selection. The tuned Bagging model (50 trees, max_features=0.5) achieved 96.49% test accuracy and an F1-score of 0.950, the tuned AdaBoost model (200 estimators, learning rate 1.0) achieved 97.37% test accuracy and an F1-score of 0.963, and the tuned Stacked Ensemble (SVM and Decision Tree base learners with a Logistic Regression meta-learner) achieved 96.49% test accuracy, an F1-score of 0.950, and the highest ROC-AUC of 0.996. The experiment empirically demonstrates how bagging targets variance, boosting targets bias, and stacking exploits model diversity, and confirms that all three ensembling strategies clearly improve on a single base learner for this classification task.

13 References

1. Scikit-learn Developers, "Ensemble methods," scikit-learn documentation. [Online]. Available: scikit-learn.org/stable/modules/ensemble.html
2. Scikit-learn Developers, "Bagging meta-estimator," scikit-learn documentation. [Online]. Available: [scikit-learn.org: Bagging meta-estimator](http://scikit-learn.org/stable/modules/ensemble.html#bagging-meta-estimator)
3. Scikit-learn Developers, "Stacked generalization," scikit-learn documentation. [Online]. Available: [scikit-learn.org: Stacked generalization](http://scikit-learn.org/stable/modules/ensemble.html#stacked-generalization)

4. W. N. Street, W. H. Wolberg, and O. L. Mangasarian, “Nuclear feature extraction for breast tumor diagnosis,” in IS&T/SPIE International Symposium on Electronic Imaging, 1993.
5. UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set.” [Online]. Available: <https://archive.ics.uci.edu>
6. Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
7. J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
8. D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.